work at the login endpoint. You can also return a useful WWW-Authenticate header if the token has expired. Listing 5.4 shows the updated methods. Update the implementation and save the file.

Listing 5.4 Parsing Bearer Authorization headers

```
public void validateToken(Request request, Response response)
{
    var tokenId = request.headers("Authorization");    #A
    if (tokenId == null || !tokenId.startsWith("Bearer "))
{    #A
        return;
    }
    tokenId = tokenId.substring(7);    #B

    tokenStore.read(request, tokenId).ifPresent(token -> {
        if (Instant.now().isBefore(token.expiry)) {
            request.attribute("subject", token.username);
            token.attributes.forEach(request::attribute);
        } else {
            response.header("WWW-Authenticate",    #C
                    "Bearer error=\"invalid_token\"," +    #C

"error_description=\"Expired\"");    #C
         halt(401);
        }
    });
}
public JSONObject logout(Request request, Response response)
{
    var tokenId = request.headers("Authorization");    #A
    if (tokenId == null || !tokenId.startsWith("Bearer "))
{    #A
        throw new IllegalArgumentException("missing token
header");
    }
```